DEGREE PROJECT IN MATHEMATICAL STATISTICS
SECOND CYCLE, 30 CREDITS
STOCKHOLM, SWEDEN 2021

Deep Bayesian Neural Networks for Prediction of Insurance Premiums

Probabilistic Deep Learning

Nils Olsgärde

KTH ROYAL INSTITUTE OF TECHNOLOGY
SCHOOL OF ENGINEERING SCIENCES

Author

Nils Olsgärde, nilsol@kth.se
Applied and Computational Mathematics
KTH Royal Institute of Technology

Place for Project

Söderberg & Partners Insurance Consulting AB (SPICAB)
Stockholm, Sweden

Examiner

Pierre Nyquist
Mathematical Statistics
KTH Royal Institute of Technology

Supervisors

Viktor Nilsson
Mathematical Statistics
KTH Royal Institute of Technology

Otto Åkerström
Non-Life
Söderberg & Partners

Abstract

In this project, the problem concerns predicting insurance premiums and particularly vehicle insurance premiums. These predictions were made with the help of Bayesian Neural Networks (BNNs), a type of Artificial Neural Network (ANN). The central concept of BNNs is that the parameters of the network follow distributions, which is beneficial. The modeling was done with the help of TensorFlow's Probability API, where a few models were built and tested on the data provided. The results conclude the possibility of predicting insurance premiums. However, the output distributions in this report were too wide to use. More data, both in volume and in the number of features, and better-structured data are needed. With better data, there is potential in building BNN and other machine learning (ML) models that could be useful for production purposes.

Keywords

Master Thesis, Artificial Intelligence, Artificial Neural Networks, Bayesian Neural Networks, Insurance Premiums, Vehicle Insurance

Djupa Bayesianska neurala nätverk för prediktioner på fordonsförsäkringar

Probabilistisk djupinlärning

Abstract

Detta projekt grundar sig i möjligheten till att predikera försäkringspremier, mer specifikt fordonsförsäkringspremier. Prediktioner har gjorts med hjälp av Bayesianska neurala nätverk, vilket är en typ av artificiella neurala nätverk. Det huvudsakliga konceptet med Bayesianska neurala nätverk är att parametrarna i nätverket följer distributioner vilket har vissa fördelar och inte är fallet för vanliga artificiella neurala nätverk. Ett antal modeller har konstruerats med hjälp av TensorFlow Probability API:t som tränats och testats på given data. Resultatet visar att det finns potential att prediktera premier med hjälp av de egenskapspunkter[†] som finns tillgängliga, men att resultaten inte är tillräckligt bra för att kunna användas i produktion. Med mer data, både till mängd och egenskapspunkter samt bättre strukturerad data finns potential att skapa bättre modeller av intresse för produktion.

Nyckelord

Masterexamensarbete, Artificiell intelligens, Artificiella neurala nätverk, Bayesianska neurala nätverk, försäkringspremier, fordonsförsäkringar

[†] "Features" på engelska

Acknowledgements

To begin with, I want to extend my sincere gratitude to SPICAB for giving me this opportunity, with a special thanks to my manager at the time of this thesis, Niklas Martinsson. Further, I would like to extend a special thanks to my supervisor at SPICAB, Otto Åkerström, for the great discussions and for the feedback that has help me through this project. For the go ahead and the opportunity from KTH, I would like to thank the examiner, Pierre Nyquist. One more special thanks to my supervisor at KTH, Viktor Nilsson, for discussions and the feedback I have received during the project. My last acknowledgment goes out to all friends, family, and colleagues that have helped me in small and major ways during this term. This thesis work has been a real pleasure and a great opportunity, again a sincere thanks to the stakeholder and the supervisors.

Acronyms

ANN	Artificial Neural Network
AI	Artificial Intelligence
FFNN	Feed Forward Neural Network
BNN	Bayesian Neural Network
VBM	Variational Bayesian methods
ELBO	evidence lower bound
LLN	law of large numbers
CLT	central limit theorem
MFA	mean-field approximation
EM	Expectation-Maximization
MCMC	Markov chain Monte Carlo
ML	machine learning
API	Application Programming Interface
BI	Business intelligence
mse	mean squared error

Contents

1 Introduction **1**

1.1 Background 1

1.2 Problem 1

1.3 Goal 1

1.4 Stakeholder 2

1.5 Related Work 2

1.6 Delimitations 2

1.6.1 Research Questions 2

1.7 Outline 3

2 Background **4**

2.1 Artificial Neural Networks 4

2.1.1 Feed Forward Neural Networks 4

2.2 Uncertainties 5

2.2.1 Aleatoric Uncertainty 5

2.2.2 Epistemic Uncertainty 6

2.3 Bayesian Statistics 6

2.3.1 Bayes' Theorem 6

2.3.2 Credible Interval 7

2.4 Bayesian Neural Networks 7

2.4.1 Variational Bayesian Methods 8

2.4.2 Monte Carlo Methods 11

2.4.3 Loss function 11

2.4.4 TensorFlow Probability 12

3 Method and Set-up of problem **15**

3.1 Familiarizing with TensorFlow Probability 15

3.1.1 Toy example 15

3.2 Set-up 16
3.2.1 Data 16
3.3 Model Building 18
3.3.1 Evaluation of Models 18

4 Result **19**
4.1 Toy Example 19
4.2 Insurance Premiums 20
4.2.1 Data 20
4.2.2 Building Models 24

5 Conclusions **26**
5.1 Discussion 26
5.1.1 What could have been done better? 27
5.2 Future Work 27
5.2.1 Reliable Data 27
5.2.2 Hyperparameter Tuning 28
5.2.3 Bayesian Neural Networks 28
5.3 Final Words 28

References **29**

1. Introduction

This chapter introduces the thesis project and provides insight into what the report investigates.

1.1 Background

The insurance industry is highly data-driven, and insurers rely on data to evaluate risks. The data corresponding to a customer is used to evaluate the risk and the premium. This price may vary depending on the insurer, implying that they are evaluating the risk differently. The coverage for each insurance offer may also vary; thus, picking insurance can be cumbersome, considering price and coverage. This is where a brokerage firm can help the customer receive the right coverage and price. Nevertheless, knowing what the correct premium should be is not an easy task.

Usually, there are three parties of interest in these transactions, the insurer, who takes the risk and sets the price. Then, the customer who has the interest to buy insurance for a risk. Finally, the broker works to make the best insurance decision for the customer.

1.2 Problem

The pricing of insurance, also known as the premium, can be seen as a black box, in the sense that one usually does not get any insight into how the price is set. If it is possible to predict, with high accuracy, the price given previous experience, one would be in a better position to know if the pricing is fair or not. With the algorithms and computing power of today, machine learning (ML) could possibly help brokers in their work to make better decisions for the customers. This project has explored Bayesian Neural Networks (BNNs) for predicting premiums and tries to provide valuable insights into the uncertainties of the predictions.

1.3 Goal

This project has been based on vehicle insurance premium prediction, where the data set is provided from brokered insurances. It has aimed to deliver the foundation for a

model that can predict premiums and present them appropriately. Foundation is, in this case, what data is of interest, pre-processing of the data, and insights into BNN in this setting.

1.4 Stakeholder

The thesis project was executed in collaboration with Söderberg & Partners Insurance Consulting, who also has been the main stakeholder of this project. This report can become a foundation and a possible start on their journey of ML and Artificial Intelligence (AI).

1.5 Related Work

Recent studies have been made looking at building ML models for actuaries to price insurance instead of their traditional framework of generalized linear models [7, 15]. These studies show how the industry is shifting toward this field and emphasizes the importance of brokerage firms joining this movement. One problem is the lack of transparency in regards to ML models [15]. This might be a problem for insurers but is not as big a problem for brokerage firms since they are not setting the premium.

Since data sets are not always complete, it can be of interest to present the uncertainty of predictions. This is one reason why BNNs is a promising research direction [16].

1.6 Delimitations

The project has been focused on a specific type of insurance and a specific product of the stakeholder. The underlying reason was to limit the complexity since all insurances were of the same type and have the same coverage. One important note for this project to be feasible and sustainable for the stakeholder was building the networks through TensorFlow.

1.6.1 Research Questions

- Can BNNs be implemented in the setting of predicting insurance premiums, with high accuracy and insights into the uncertainty of the predictions?

1.7 Outline

The report introduces an overview of the concept of Artificial Neural Networks (ANNs) and BNNs. Then some brief descriptions on how some of the popular probabilistic BNN layers given by the TensorFlow Probability Application Programming Interface (API) work. This background is then followed by how the work has been conducted and the results. The results are then discussed, and a few conclusions are given, including some insights into what future work could be of interest for further improvements.

2. Background

In this chapter, the mathematics and statistics that have been used are introduced. The main coding package that is related to Bayesian Neural Networks (BNNs) is also briefly presented.

2.1 Artificial Neural Networks

A simple mathematical representation of a human brain is the concept of Artificial Neural Networks (ANNs). There are many different types of ANNs; the Feed Forward Neural Network (FFNN) type has been the focus in this thesis.

2.1.1 Feed Forward Neural Networks

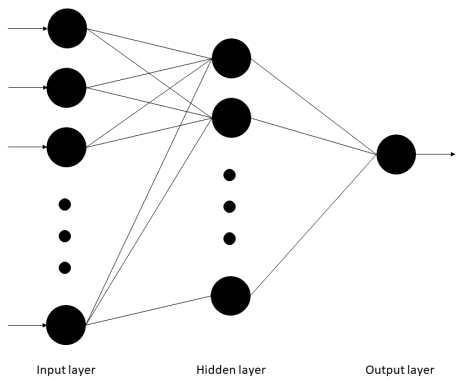


Figure 1: Four layered feed-forward neural network

FFNNs are the simplest form of neural networks, where the connections between the nodes do not form a circle. The information is only passed in one direction [5]. Each node in a hidden layer is connected with each node in the previous layer and each node in the next layer. These connections have associated weights, w, and each node also has a parameter known as the bias, b; these determine the behavior of the network. The connections into a node are added together, including bias, and passed to an activation function. Activation functions are usually nonlinear, such as a sigmoid or ReLu. The outputs of all the nodes are then fed through the connection to the next layer, which in the case of figure 1 is the output layer. These connections follow the same principle as previously described.

Let the network in figure 1 have an input layer of m nodes and n nodes in the hidden layer. Then letting $\sigma(.)$ denote an arbitrary activation function and assuming no activation function for the output layer. The following equation represents the

approximation function corresponding to this network,

$$f_n\left(x_1, \ldots, x_m\right) = \sum_{i=1}^{n} w_i \sigma\left(\sum_{j=1}^{m} w_{ij} x_j + b_i\right).$$

FFNNs can be designed with an arbitrary depth of layers, where each hidden layer has an arbitrary number of neurons. Furthermore, an important theorem is the universal approximation theorem. This theorem states that a FFNN with one hidden layer can approximate any continuous function arbitrarily well [4, 8]. In mathematical terms, the theorem is stated as follows.

Theorem 1. *Let $\sigma(.)$ be an arbitrary activation function, $X \subseteq \mathrm{R}^m$ and is compact. Let the space of continuous functions on X be denoted by $C(X)$. Then $\forall f \in C(X), \forall \varepsilon > 0 : \exists n \in \mathrm{N}, a_{ij}, b_i\ w_i \in \mathrm{R}, i \in \{1 \ldots n\}, j \in \{1 \ldots m\}$*

$$f_n\left(x_1, \ldots, x_m\right) = \sum_{i=1}^{n} w_i \sigma\left(\sum_{j=1}^{m} a_{ij} x_j + b_i\right)$$

as an approximation of the function f(.), such that

$$\|f - f_n\| < \varepsilon.$$

This theorem is the main reason for its popularity and is a feature of ANNs that is highly desirable. This usually follows the classical approach, which can be seen as an algorithmic approach rather than probabilistic [3].

2.2 Uncertainties

In many ML applications, there are uncertainties in the predictions. These uncertainties can be classified into two categories, aleatoric and epistemic.

2.2.1 Aleatoric Uncertainty

This class of uncertainty is also known as statistical uncertainty and is due to differences in unknowns that make the result differ for repeated experiments. This uncertainty is quite often referred to as an error term, ϵ. An example of this is the coin-flipping experiment where there is a stochastic component that can not be reduced by any source of additional information [9].

2.2.2 Epistemic Uncertainty

Epistemic uncertainty is instead about the uncertainty in the results due to lack of data; it can be lack of data in a region or lack of a feature that could have been included. It is also known as the uncertainty due to lack of knowledge, this type of uncertainty can potentially be reduced by additional information [9].

For this particular thesis project, estimating these uncertainties is of interest since it could make predictions more reliable. It should also be mentioned that uncertainty estimation is always of interest, but it is not straightforward how to implement it, hence the importance of Bayesian statistics.

2.3 Bayesian Statistics

In statistics, there are two main methods of inference, frequentist inference, and Bayesian inference. In frequentist inference, the notion of a probability of an event is how frequent it has been observed. The problem is that this results in a point estimate with no knowledge of the confidence of that estimate. In contrast, in Bayesian inference, one starts with a prior distribution corresponding to the prior beliefs. These beliefs are then updated by the evidence of the experiment, resulting in a posterior distribution. The posterior distribution can then be used for inference and uncertainty estimation for the predictions.

One important area of distinction between these two approaches is hypothesis testing [6]. This is illustrated by two simple ideas that the Bayesian paradigm is based on. The first idea of Bayesian statistics is that probability is a measure of belief in the occurrence of events instead of the limit in the frequency of the events when the number of samples goes towards infinity, as seen by the frequentist. The second idea is that prior beliefs influence the posterior beliefs [10].

2.3.1 Bayes' Theorem

The famous theorem summarizes these two ideas and is a very simple formula to invert conditional probabilities, and its interpretation in Bayesian statistics.

$$p(H|X) = \frac{p(X|H)p(H)}{p(X)} = \frac{p(X|H)p(H)}{\int_H p(X,H)} \tag{1}$$

The Bayesian interpretation here is that H is explicitly considered the hypothesis; for instance, the parameters in an ANN, and X is the data. The classical interpretation is that H and X are sets of outcomes [10]. $p(X)$ is called the evidence, $p(H)$ the prior, $p(X|H)$ the likelihood, and $p(H|X)$ the posterior. This theorem and interpretation can be viewed as learning from the data X. This yields that the Bayesian paradigm offers both a solid approach for the quantification of uncertainty in deep learning models and a mathematical framework to understand numerous learning strategies and regularization techniques that are used in classical deep learning [18].

2.3.2 Credible Interval

In Bayesian statistics, a credible interval is an interval for which an unknown parameter lies with some probability. The estimated parameter is viewed as a random variable, and credible intervals are analogous to confidence intervals in frequentist statistics. The credible interval has been chosen as the interval for which the mean is the central point.

2.4 Bayesian Neural Networks

The idea behind Bayesian Neural Networks (BNNs) is to get the benefits of Artificial Neural Networks (ANNs) but also from probabilistic models; one crucial aspect is that BNNs are a type of ANN. There are multiple ways to yield the benefits of both, but in this thesis, the following two approaches were used.

1. By seeing the weights of the network as random variables following probability distributions.
2. By letting the network have a distribution output instead of a point estimate. [†]

In both cases, some assumptions were needed. From the law of large numbers (LLN) and central limit theorem (CLT), the assumption is that the distributions over the weights are or are close to Gaussian. This is partially justified by Walker [25], who says that the posterior distribution will usually be approximately Gaussian for a large data set for a given positive and continuous prior density. Interestingly, as the number of data points goes to infinity, the variance of the Gaussians will approach zero in the limit [1]. It should also be mentioned that it is of computational advantage to use Gaussians.

[†]This is not precisely Bayesian, but it can be seen as probabilistic programming

For the distribution output, one usually assumes it to be Gaussian, but in the cases where the output cannot be negative, a Gaussian output was not suitable. Since insurance premiums are not negative, a more suitable distribution was the lognormal. The lognormal distribution has two parameters, $\sigma > 0$ and $\mu \in (-\infty, \infty)$, meaning that the main network output should be of dimension two.

2.4.1 Variational Bayesian Methods

BNNs are built on a selection of methods that are included in what is named as Variational Bayesian methods (VBM) and is a family of techniques for approximating intractable integrals. Below a few important concepts that are common types of VBM are presented.

Evidence Lower Bound

One important part in VBM is the evidence lower bound (ELBO), also known as the variational lower bound, that holds an essential role in the VBM derivations for BNN. With the help of equation (1) for the posterior, starting with the logarithm of the evidence and with the help of Jensen's inequality, the ELBO can be derived as

$$\begin{aligned}
\log p(X) &= \log \int_H p(X, H) \\
&= \log \int_H p(X, H) \frac{q(H)}{q(H)} \\
&= \log \mathbb{E}_q \left[\frac{p(X, H)}{q(H)} \right] \\
&\geq \mathbb{E}_q \left[\log \frac{p(X, H)}{q(H)} \right] \\
&= \overbrace{\mathbb{E}_q \left[\log p(X, H) \right] - \underbrace{\mathbb{E}_q \left[\log q(H) \right]}_{\text{Entropy}}}^{\text{ELBO, } \mathcal{L}(q)} .
\end{aligned} \tag{2}$$

The right hand side of equation (2) is the ELBO, where $q(H)$ can be viewed as an arbitrary distribution to approximate the actual posterior distribution. This distribution is the so-called variational distribution. The last part is the entropy of q. This derivation shows that to maximize the marginal probability, one can instead maximize the ELBO.

Kullback–Leibler divergence

Named after Solomon Kullback and Richard A. Leibler, the relative entropy, more formally known as KL divergence, is denoted by D_{KL}. It is a measure of how one probability distribution q differs from another p, where if the q and p are identical in distribution, the KL divergence will be zero.

Definition 1. *Let $p(x)$ represent the true distribution of data and $q(x)$ the approximation of $p(x)$, then the KL divergance is defined by*

$$D_{KL}(q(x)||p(x)) = \int_{-\infty}^{\infty} q(x) \ln \frac{q(x)}{p(x)} dx.$$

By the definition of the KL divergence for continuous distributions, it can be written for the arbitrary distribution $q(H)$ and the true posterior distribution $p(H|X)$ as

$$\begin{aligned} D_{KL}(q(H)||p(H|X)) &= \mathbb{E}_q \left[\log \frac{q(H)}{p(H|X)} \right] \\ &= -\mathbb{E}_q \left[\log \frac{p(H|X)}{q(H)} \right] \\ &= -\left(\mathbb{E}_q \left[\log \frac{p(X,H)}{q(H)} \right] - \mathbb{E}_q \left[\log p(X) \right] \right) \\ &= -\underbrace{\left(\mathbb{E}_q \left[\log p(X,H) - \log q(H) \right] \right)}_{\text{ELBO, } \mathcal{L}(q)} + \log p(X). \end{aligned}$$

By moving the terms, the equation that is used in many Bayesian programming situations is found as

$$\log p(X) = \mathcal{L}(q) + D_{KL}(q(H)||p(H|X)). \tag{3}$$

An intuitive view of the KL divergence, for the important cases are given in a lecture by Willie Neiswanger [17].

- If q is high and p is high, then we are happy (i.e. low KL divergence).
- If q is high and p is low then we pay a price (i.e. high KL divergence).
- If q is low then we don't care (i.e. also low KL divergence, regardless of p).

Mean-Field Approximation

In mean-field approximation (MFA) it is assumed that the unknown variables can be partitioned into independent parts. By KL divergence, one can derive mutually independent functions that will define the shape of q. The functional representation of q will usually take the form of well-known distributions, leading to an iterative algorithm similar to the Expectation-Maximization (EM) algorithm [11].

To understand the MFA one first has to notice that the logarithm of the evidence, $\log p(X)$, in equation (3) is constant. To find the best variational distribution, q, one would like to minimize the KL divergence. This will be the result if the ELBO is maximized.

This is done as mentioned before by factorizing the variational distribution over the latent variables; the resulting equation is a generic member of the mean-field variational family.

$$q(\theta_1, \ldots, \theta_m) = \prod_{j=1}^{m} q(\theta_i) \tag{4}$$

Looking at the variational distribution over a single latent variable, $q(\theta_i)$, it is referred to as the *local variational approximation*. Here H is represented as the parameters θ_i's. In the optimization, these are chosen to maximize the ELBO of equation (2). It should also be emphasized that the variational family does not depend on the data, which can be seen in equation (4). However, the data comes in with the ELBO, see equation (2), which connects the data with the fitted variational density [2].

Feasibility

A problem that often comes up with Bayesian inference is computing the posterior distribution, which is intractable to calculate analytically. Furthermore, for a nontrivial neural network, using Markov chain Monte Carlo (MCMC) is too slow to be feasible, but MFA can be used to make it computationally practical [14]. It should be noted that MFA requires analytical solutions for the expectations with respect to the approximate posterior. This is generally intractable [13], for this reason other methods are used, see section 2.4.4 for more information.

2.4.2 Monte Carlo Methods

Monte Carlo methods are a class of computational methods used for approximations; the class relies on random sampling. There are mainly three problem types that the class is used for: optimization, numerical integration, and generating draws from probability distributions. In this project, the primary type was the numerical integration, which all the probabilistic layers built by TensorFlow use. The Monte Carlo approximation for integration of a function $f(x)$ is given by,

$$I = \frac{1}{n}\sum_{i=1}^{n} f(x_i).$$

Where $\{x_i\}_{i=1}^{n}$ are n independent samples of a suitable random variable X, i.e. uniformly distributed on the integration domain. This approximation will result in an unbiased estimate and can be used to estimate an expectation.

2.4.3 Loss function

When training an Artificial Neural Network (ANN), a function representing how correct the network is needs to be specified. That function is commonly known as the loss function, which can be chosen in many ways. The choice is based on what the network is used for; since the networks in this thesis are regression networks and with a distribution output, the common choice is the negative log-likelihood.

Negative Log-Likelihood

This loss function can be viewed as how probable the correct value is according to the output distribution. If the distribution output is centered around the correct output, the probability will be high. This will in return make the log-likelihood close to zero, since $\log(1) = 0$ but if it is not probable the logarithm will be very negative since $\lim_{z\to 0}\log(z) = -\infty$. For this reason, the negative log-likelihood is a good loss function when the output is a distribution. It should be noted that this likelihood is not the same as the one mentioned in section 2.3.1. The likelihood in this loss function is instead $p(Y|X, H)$, which is the density of the output given the parameters of the network and the data corresponding to the output. In Python it can be expressed as,

```
negloglik = lambda y, rv_y: -rv_y.log_prob(y).
```

Where `y` is the label and `rv_y` is the output distribution of the model.

Another metric that measures how well a model performs is the mean squared error (mse), it measures the average of the squares of the errors, in mathematical terms as,

$$\text{mse} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2.$$

Where $\hat{y}_i$ is the prediction of y_i, and n is the number of samples in the set.

2.4.4 TensorFlow Probability

TensorFlow Probability is a library for many different probabilistic and statistical tools built upon TensorFlow. In this library, an integration of probabilistic methods with deep neural networks is provided; thus, it was used in this project. In this section, a few layers of interest are presented, starting with the distribution layer that makes the output a distribution instead of a point estimate. This is followed by the layers built on the popular layer dense, the common Feed Forward Neural Network (FFNN) layer. The last layer presented in this section is the Flipout-layer, which is the layer that ultimately got used in this thesis.

DistributionLambda

This layer enables distributions as output; the layer takes in the output of a network and uses it in a functional manner to represent it as a distribution. The layer is defined by a user-given function that returns a distribution, often with the help of the distributions provided by TensorFlow [20]. When a Gaussian is used as the output distribution, the network feeds two outputs into this layer, which then uses these to represent a Gaussian distribution. The network is trained to estimate the mean and variance of the output given the data. If the distribution output instead is lognormal, the network should still feed it two outputs, the distribution parameters. These parameters define the distribution, so the network trains on estimating the distribution given the data.

DenseReparameterization

This layer uses a trick called the reparameterization trick making it possible to rewrite the expectation with respect to the parameterized distribution of interest. Then the Monte Carlo approximation of this expectation is differentiable with respect to the parameter ϕ. The trick can be summarized as follows, let $\boldsymbol{\theta}$ be a continuous random variable such that $\boldsymbol{\theta} \sim q_\phi(\boldsymbol{\theta}|\mathbf{x})$, it is then possible to express random variable $\boldsymbol{\theta}$

as a deterministic variable, i.e. $\boldsymbol{\theta} = g_\phi(\boldsymbol{\epsilon}, \mathbf{x})$. For which $\boldsymbol{\epsilon}$ is an auxiliary variable following the independent distribution $p(\boldsymbol{\epsilon})$ and $g_\phi(\cdot)$ is some vector-valued function parameterized by ϕ [13, 23].

This method is a method in the class of methods called "weight perturbation", where the weights, $\boldsymbol{\theta}$, of a neural network are sampled stochastically at training. Let $\theta_{ij} \sim \mathcal{N}(\bar{\theta}_{ij}, \sigma_{ij})$, then by the reparameterization trick it can be written as $\theta_{ij} = \bar{\theta}_{ij} + \sigma_{ij}\epsilon_{ij}$, where $\epsilon_{ij} \sim \mathcal{N}(0, 1)$ and $\bar{\theta}_{ij}$ is the mean of that weight. In this representation the second term is called the perturbation and follows the perturbation distribution. This representation makes it possible for the gradients to be computed using backpropagation [26].

DenseLocalReparameterization

This layer is simply another iteration on the reparameterization layer, where the estimator that performs a Monte Carlo approximation is now based on the local reparameterization estimator [22]. (For more information on this estimator, refer to [13].) This layer deviates a bit from the parameterization done in [13], by having both the scale and the location, otherwise known as mean and deviation, of the distributions being trainable. This is called additive noise parameterization, read more about this in [12].

DenseFlipout

DenseFlipout is a functional dense layer in TensorFlow similar to the standard dense layer, but with the difference that it implements Bayesian variational inference, such that the weights are drawn from distributions. The unique thing with this layer is that it uses the Flipout estimator, which performs a Monte Carlo approximation of the distribution integrating over the weights [21].

Both of the reparameterization methods are suffering from a problem relating to using the same perturbation for all training examples in a mini-batch. The problem results in a more considerable variance of the gradient estimates; this variance results from the correlation between the gradients induced using the same perturbation. The method used by this layer, the Flipout estimator, is "an efficient way to perturb the weights quasi-independently within a mini-batch" as mentioned in the article [26]. There are two assumptions about the distribution, q_ϕ, which the weights follow.

- The perturbations of different weights are independent.
- The perturbation distribution is symmetric around zero.

By these assumptions, the distribution is invariant to element-wise multiplication by a random sign matrix; as in the article, element-wise multiplication is denoted by $\circ$.

Observation 1. *Let q_ϕ be a perturbation distribution that satisfies the two assumptions above, let $\widehat{\Delta\boldsymbol{\theta}} \sim q_\phi$ be the perturbation. Now let S be an independent random sign matrix, then $\Delta\boldsymbol{\theta} = \widehat{\Delta\boldsymbol{\theta}} \circ S$ is identically distributed to $\widehat{\Delta\boldsymbol{\theta}}$. Not only that, but the loss gradients computed using $\Delta\boldsymbol{\theta}$ are identically distributed to those computed using $\widehat{\Delta\boldsymbol{\theta}}$.*

The Flipout estimator uses this fact by having the same perturbation $\widehat{\Delta\boldsymbol{\theta}}$ as a base for all examples in a mini-batch, but multiplying it by a different rank-one sign matrix for each example. This is done as,

$$\boldsymbol{\theta}_i = \widehat{\Delta\boldsymbol{\theta}} \circ r_i s_i^\intercal.$$

In this notation the subscript represents ith example in the mini-batch, r_i and s_i are are random vectors whose entries are sampled uniformly from ± 1. By observation 1 the gradients computed will be identically distributed to those computed using shared weight perturbations. For this reason, the loss gradients will be unbiased for the Flipout estimator. Because the decorrelation of the gradients between different training examples, it can achieve much lower variance updates when averaging over a mini-batch.

3. Method and Set-up of problem

In this part, the method used and the set-up of the problem are presented.

3.1 Familiarizing with TensorFlow Probability

This thesis project focused on using the probability API that TensorFlow provides; it became essential to get familiar with how to use it and get some insights into the benefits of using it. An essential benefit is that TensorFlow is well-used, which means that there are many examples to find. Since this API provides a layer that outputs a distribution, the $\mathrm{DistributionLambda}$ layer has an associated quantile method. With this method, it is possible to get the credible interval of interest.

3.1.1 Toy example

As a start to get familiar with probabilistic programming using the provided API, the project initially focused on a toy example. I.e., an example where the dependency was known, and one could visually see how the predictions behaved. In this thesis, the following dependence was used for the toy example,

$$y = x + \frac{3}{10}\sin 2\pi(x+\epsilon) + \frac{2}{10}\sin 4\pi(x+\epsilon) + 2\epsilon.$$

Where $\epsilon \sim N(0, 0.02)$, which gives the data set, that can be seen in figure 2. For the training, validation, and testing, the data was generated in the interval from 0.5 to 1. For visual inspection, the interval was set from 0.25 to 1.25. This to find out how the model predicts samples that are not in the training interval; it was assumed that the models could not predict the curvature that happens outside the training interval, see figure 2.

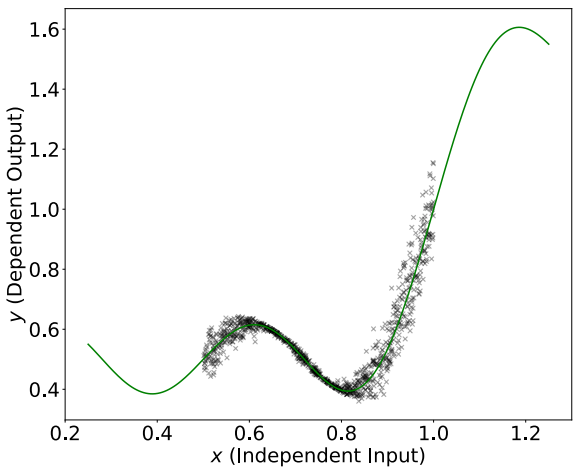


Figure 2: The known dependence

A few different models/ architectures were tested, but only with the same number of nodes in each layer. It should also be noted that there was always an output layer that outputs the predicted parameters to the distribution; this layer was of the same type as the rest of the model. In the toy

example, the distribution output was Gaussian, for which the output layer will have two nodes.

All models also ran with callback, an object that can be used when training a model. It can, for example, periodically save the model to disk or be used for early stopping to stop the training if some criteria are satisfied. The latter was used in this thesis project, which hopefully ensured better convergence and prevented unwanted overfitting.

3.2 Set-up

The aim of this project was to predict vehicle insurance premiums; the predictions were made on one insurance product where one can assume the coverage to be constant throughout the years. The label of this regression problem is the insurance premium, which was assumed to be dependent on the vehicle's features.

3.2.1 Data

For this thesis project, the stakeholder provided the data. The data consisted of features of the vehicles and the corresponding yearly premiums. The features that were provided can be seen in table 1.

Feature	Type of feature	Number of distinct	Examples
Brand	Categorical	37	Audi, BMW, Volvo
Vehicle category†	Categorical	4	M1, M1G, N1, N1G
Body type ‡	Categorical	9	AA, AB, AC
Year of production	Numeric	-	2000, 2005, 2020
Insurance year	Numeric	-	2007, 2009, 2021
Engine power	Numeric	-	50, 150, 200 [kW]
Type of fuel	Categorical	3	Diesel, Petrol, Methane
Transmission	Categorical	4	Automatic, Manual
Max number of passengers	Numeric	-	2, 3, 5, 7
Length	Numeric	-	2400, 3000 [mm]
Width	Numeric	-	1500, 1800, 2000 [mm]
Height	Numeric	-	1550, 1670, 2200 [mm]
Total weight	Numeric	-	1200, 1500, 3000 [kg]
Displacement††	Numeric	-	1000, 1500, 2200 [cm^3]
Area ‡‡	Categorical	84	10, 11, 15, 77
Max speed	Numeric	-	200, 250 [km/h]

Table 1: Features

†A division of vehicles based on the purpose of the vehicle[24]
‡A code describing the vehicle body (vehicle design)[24]
††Engine size, cylinder volume
‡‡Primary location of the company that has the insurance

Data Pre-processing

Before the data was used, it needed to be processed. The first part of the processing was mapping categorical data to coded indices. This so it could be presented without giving out explicit information, which was a requirement from the stakeholder. The categorical features were then mapped to one-hot vectors such that they could be used in neural networks. For most of the numerical features, the data was scaled such that it could be presented. All features were then standardized, which means that they were centered around zero and then scaled by their deviation. This, because all features should be in the same range of values to work appropriately with a neural network. Two features that were combined in the pre-processing step were the production year and the insurance year, as the age of the vehicle at the start of the insurance year.

In this step, all rows that lacked any of the features specified in table 1 were removed to ensure that all data had all features. It should be mentioned that some features were removed for reasons such as, too many rows missing a particular feature or a feature was not of interest. For example, the feature vehicle status was not of interest since the present status is not interesting when looking at features for old insurances. This relates to how the data is saved; more on this problem can be found in section 5.2.1.

Features that could be of high importance and not included in the data-set are the history of incidents, damages, and claims. It could be of high importance since the premium is often increased at renewal if more damage was done than predicted. Unfortunately, this data was not included in the given data set.

Data Inspection

When the data inspection was done, the distributions of the features were inspected. For the regions where the data was scarce, one could assume the models to be uncertain in the predictions, resulting in a larger credible interval. When inspecting the distributions, outliers were identified and removed.

Data Split

When training, validating and testing a model, it is vital to split the data to not validate or test with the same data as the model used for training. Otherwise, the result should not be trusted; for this reason, the data was split into three parts, training, validation, and testing. In the training of the models, the training set was used for the fit and the

validation set for validating the fit. Therefore, the training could be stopped early if the network had converged; the convergence was seen on the validation set. The third part is the test set, which is the last part and was used to evaluate the models.

3.3 Model Building

Building neural network models with TensorFlow is relatively straightforward, and that is also the case for BNNs when using the TensorFlow Probability API. For that reason, it was possible to test different architectures, i.e., how many layers, nodes per layer, and types of layers. There are many other hyperparameters, so it was unfeasible to test them all.

3.3.1 Evaluation of Models

The different models that were being tested were evaluated by the loss, KL-divergence, and mean squared error (mse) on the test set. Then by a visual inspection by plotting the predictions against the labels. If the predictions were close to the labels, they would create a straight line. Since the predictions are distributions, it was also possible to plot a credible interval.

4. Result

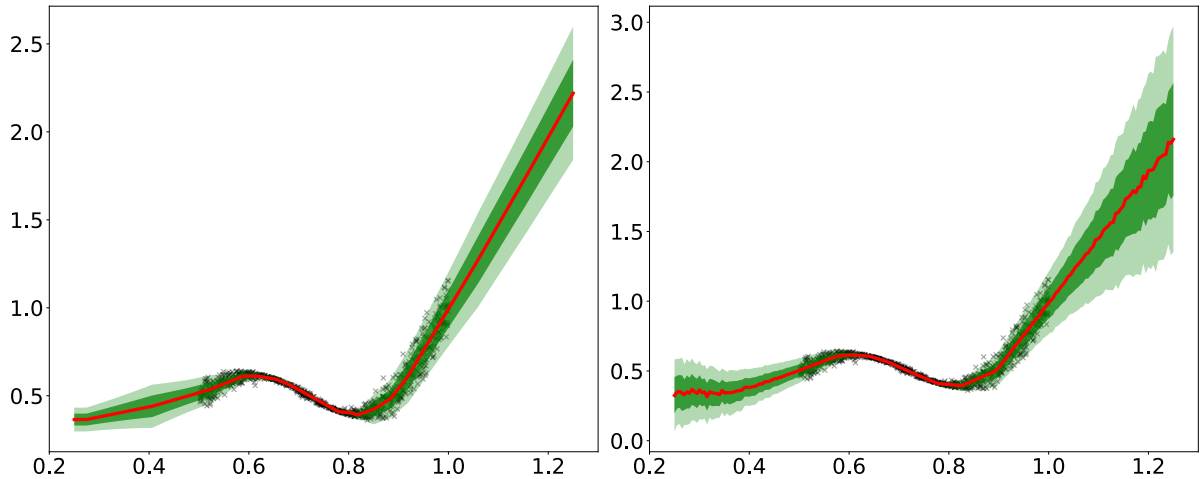
In this chapter the results from the toy example are presented followed by the results on predictions of premiums.

4.1 Toy Example

The models that were tested on this toy example and their results are as follows.

Model #	Type of Layers	# of Layers	# of Nodes	# of Parameters	negloglik	KL-div	mse
1	Dense	1	1	6	-1.1751	0.3800	0.0674
2	Dense	1	10	42	-1.0079	0.1976	0.0457
3	Dense	1	50	202	-2.7664	0.0032	0.0018
4	Dense	2	1	8	-0.5882	0.0443	0.0383
5	Dense	2	10	152	-2.8053	0.0047	0.0016
6	Dense	2	50	2752	-2.8373	-0.0042	0.0017
7	DenseFlipout	1	1	9	-0.9835	0.0789	0.0409
8	DenseFlipout	1	10	72	-2.5329	-5.7732e-4	0.0027
9	DenseFlipout	1	50	352	-2.5505	0.0048	0.0024
10	DenseFlipout	2	1	12	-0.9774	0.0431	0.0350
11	DenseFlipout	2	10	282	-2.7212	-0.0043	0.0016
12	DenseFlipout	2	50	5402	-2.5293	0.0057	0.0021

Table 1: Results of a few models that were tested



(a) Model # 6. (b) Model # 12.

Figure 3: Prediction on the visual interval, red line is the mean, dark green is the 68% credible interval and the light green is the 95% credible interval.

The Flipout model shown in figure 3b is the mean of a hundred predictions with the model; that is the reason for the trembling mean line and the boundaries. This results from the distributions over the weights, which makes it similar to an ensemble of

models. As shown in Appendix A, the dense models could also produce larger tails. However, they were not as consistent on having wider tails.

The following shows the behavior of the metrics for one of the fits of model 12.

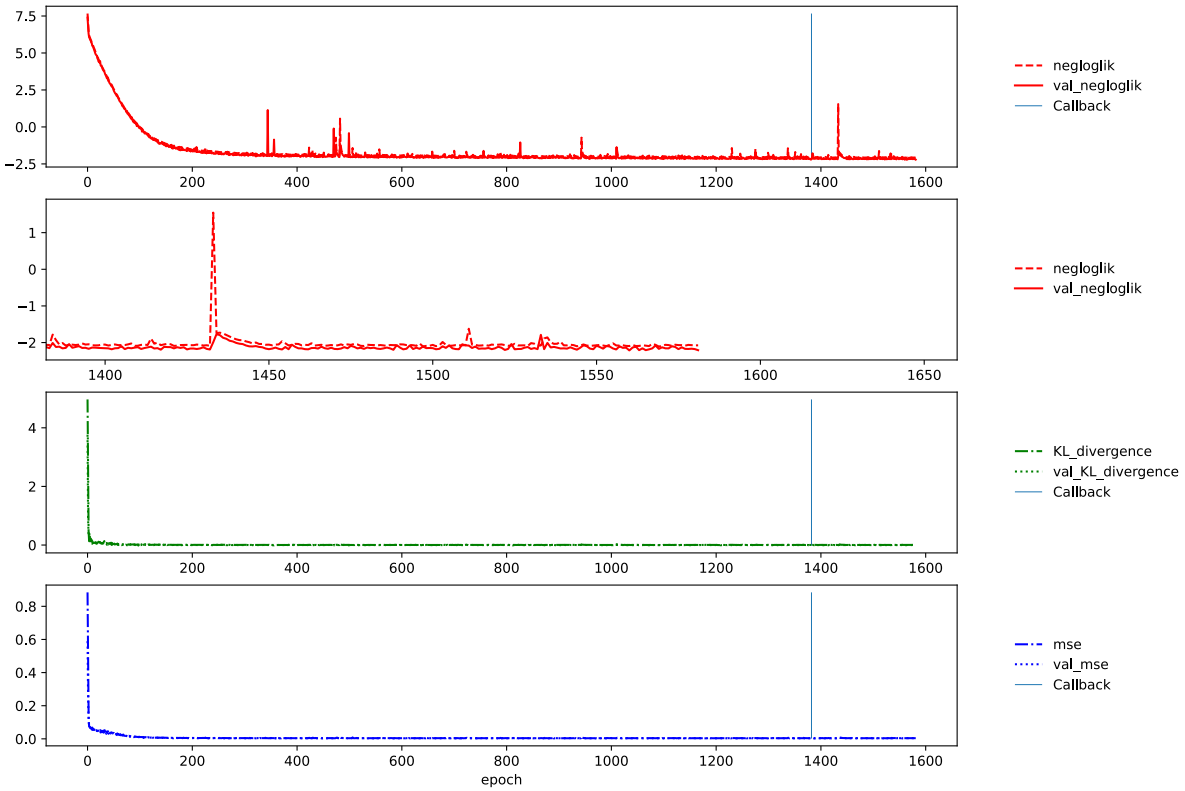


Figure 4: History of the metrics over the epochs, of model 12.

4.2 Insurance Premiums

In this section, the results from the insurance premium predictions are presented.

4.2.1 Data

The resulting data set was after some preprocessing to make it presentable for the thesis, 15 labels, and 16979 unique rows. This amount of data was a subset of the complete set but the set with the highest quality of data.

Data Inspection

The following plots show how the features are distributed.

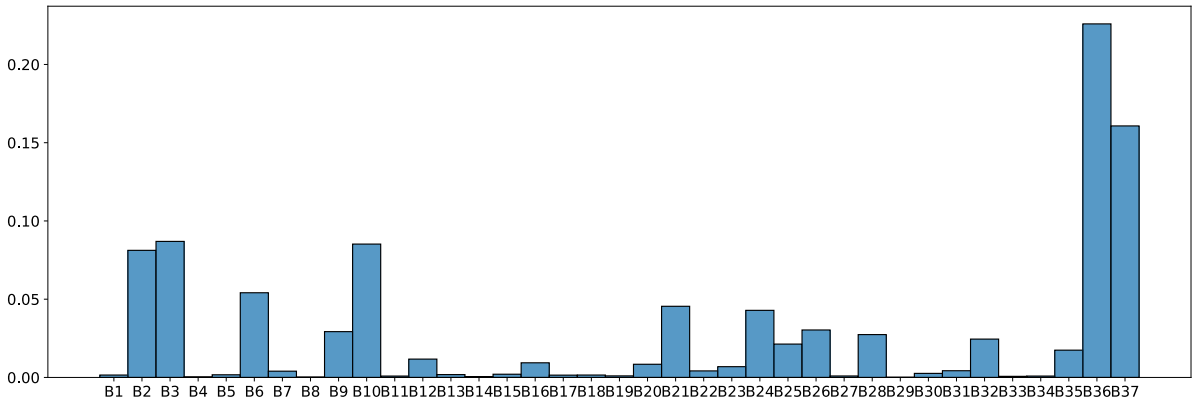
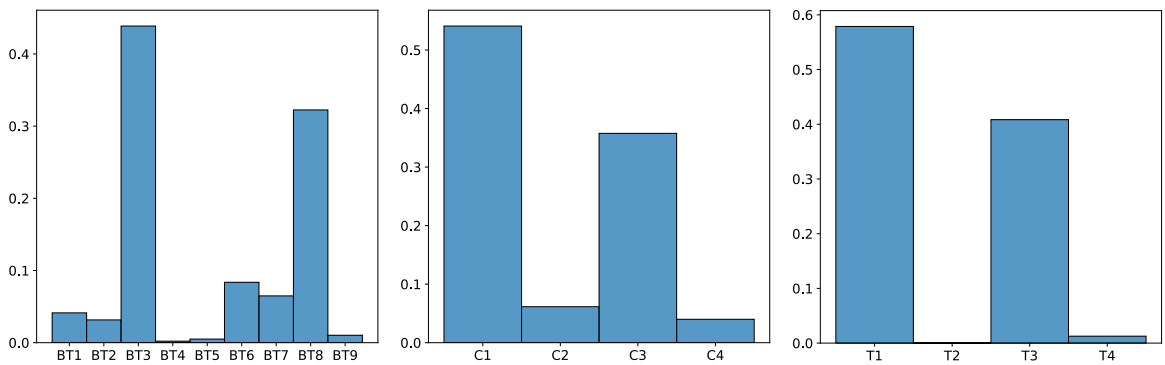
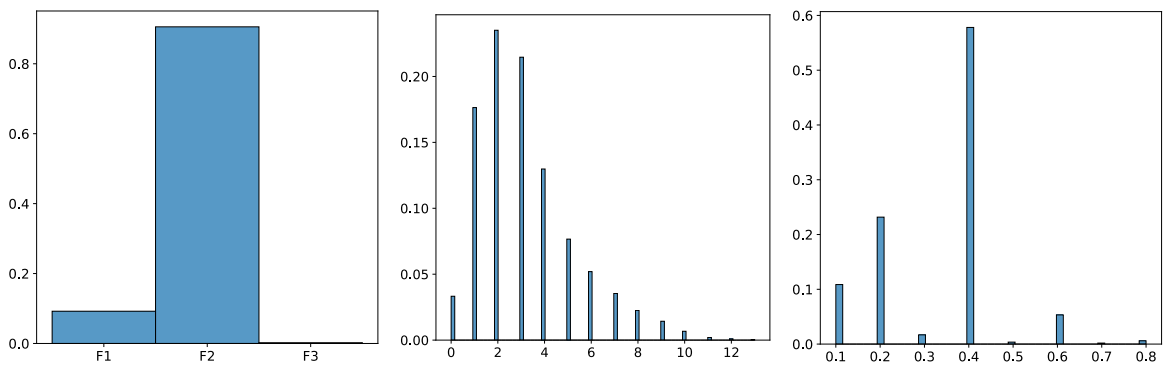


Figure 5: Distribution of brands



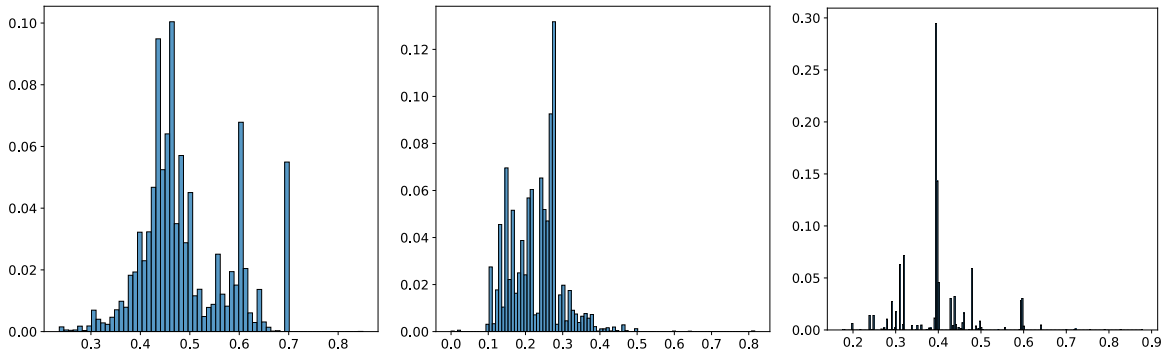
(a) Body type (b) Vehicle category (c) Transmission

Figure 6: Three feature distributions



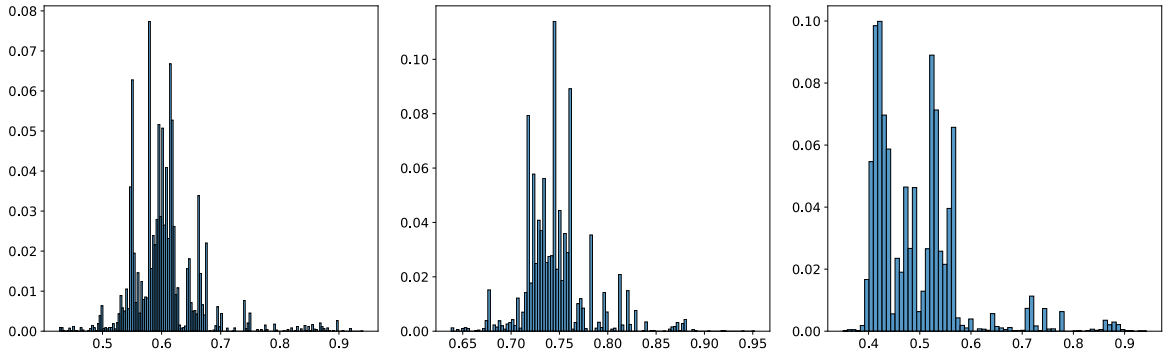
(a) Type of fuel (b) Vehicle age (c) Max number of passengers

Figure 7: Three feature distributions



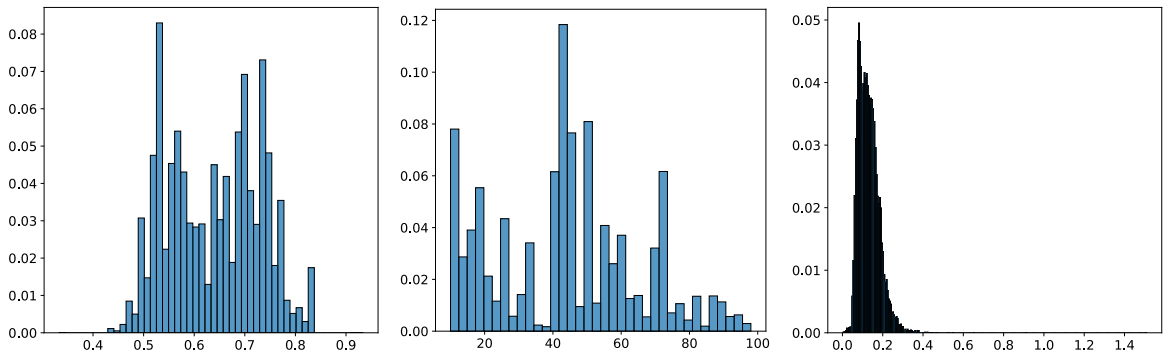
(a) Total weight (b) Engine power (c) Displacement

Figure 8: Three feature distributions



(a) Length (b) Width (c) Height

Figure 9: Three feature distributions



(a) Max speed (b) Area (c) Premium

Figure 10: Two feature distributions and the premium distribution

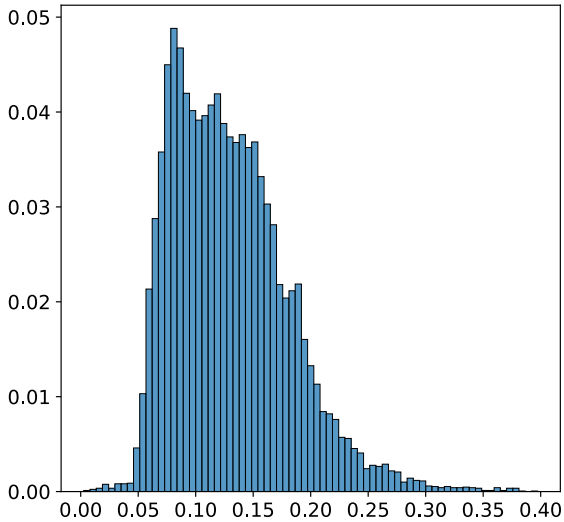


Figure 11: Distribution of the premiums after removal of outliers

The distributions show that the data was not evenly distributed over the categories nor the numeric labels. In figure 10c, some premiums deviate quite a bit from the leading group since the plot goes from zero to 1.4 on the x-axis. The insurances corresponding to the premiums with values higher than 0.4 were therefore seen as outliers and removed. There were only 17 of these outliers, so the resulting data set had 16962 unique entries. The resulting distribution of the premiums can be seen in figure 11.

The matrix in figure 12, shows the correlation between the numerical features and the label. It shows that there is some correlation between the premium and the numerical features. The correlation with categorical features was not feasible to calculate appropriately, so these features were omitted from the correlation matrix.

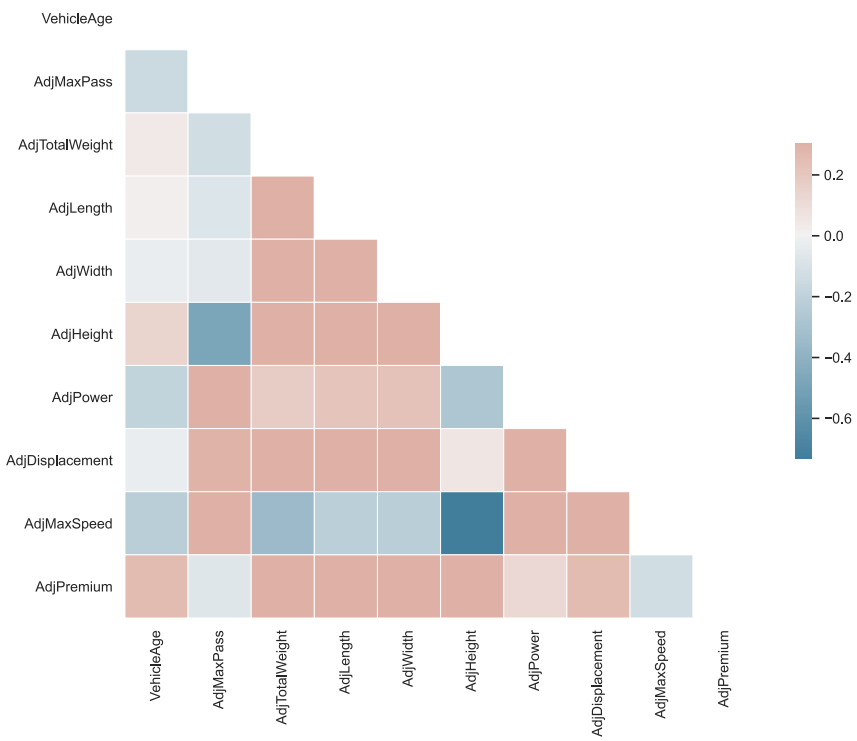


Figure 12: Correlation matrix of the numerical features

Preprocessing

As some preprocessing had been done, the categorical features needed to be mapped to one-hot vectors to be interpreted by a neural network in a correct manner. The total feature space after this mapping was of dimension 150; this was the input to the neural networks. The data was subsequently split into three parts, training which was 70% of the data, validation 29%, and testing 1%. The reason for the small test-set is so that the resulting prediction figures are not too cluttered.

Then as a final preprocessing step, the data was standardized; this was done with scikit learn's preprocessing function StandardScaler [19]. The scaler was fitted on the training data; that scaler then transformed all data sets.

4.2.2 Building Models

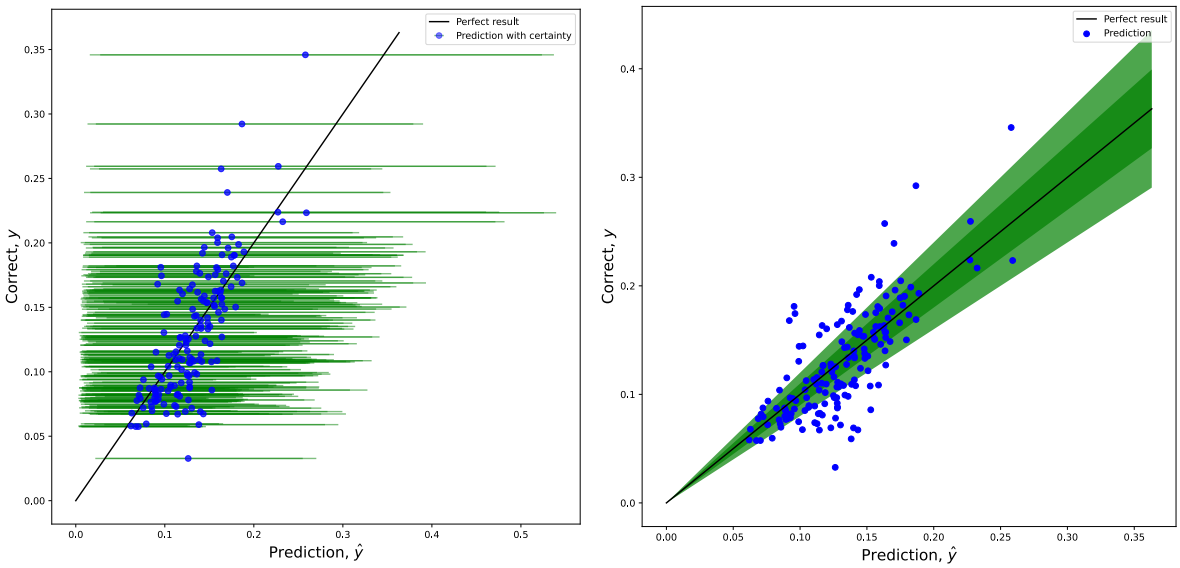
Not all combinations of hyperparameters could be tested; only some aspects of the architecture of the models was evaluated. This involved how many layers, number of nodes in each layer, other hyperparameters, such as learning rate, number of epochs, and patience of the callback were set. All models were trained for a maximum of 3000 epochs, with callback patience of 300 and a learning rate of 10^{-4}. The callback looked at the loss on the validation set, and then the weights were restored to the best, i.e., 300 epochs earlier if the training were callbacked. This was to ensure convergence and try to prevent overfitting. Below are the results of one run of each architecture.

Model #	Layer 1	Layer 2	Layer 3	Layer 4	Layer 5	# of Parameters	negloglik	KL-div	mse	Time [sec]	Epochs
1	10	-	-	-	-	3,052	-1.8578	0.0109	0.0018	839	1544
2	100	-	-	-	-	30,502	-0.9711	0.0084	0.0036 †	1698	3000
3	10	10	-	-	-	3,262	-1.8609	0.0087	0.0020	2582	1851
4	100	100	-	-	-	50,602	-0.5181	0.0113	0.0048	2569	3000
5	10	10	10	-	-	3,472	-1.9648	0.0087	0.0023	1790	2124
6	100	100	100	-	-	70,702	-0.6941	0.0076	0.0162	3294	3000
7	10	10	10	10	-	3,682	-1.9690	0.0120	0.0025	1515	1511
8‡	100	100	100	100	-	90,802	0.1404	0.0050	0.0105	2022	1691
9	10	10	10	10	10	3,892	-1.9062	0.0085	0.0018	1554	1356
10‡	100	100	100	100	100	110,902	0.6897	0.0084	0.0023	1999	1549
11	20	20	20	20	20	9,382	-1.7435	0.0090	0.0021	2098	1998

Table 2: Results of the models that were tested on the test set

† The test and validation data gave a mse of infinity
‡ Diverged, then callbacked, therefore not 3000 epochs

A few models were tested, and as presented, one of the best results was from one of the simpler models, i.e., model #5. This model has one of the lowest losses, KL-divergence close to zero and low mean square error. Below are the resulting predictions of model #5 on the test set and the history over the epochs. In appendix B the same plots are shown but for model #11, and the results are similar.



(a) Prediction with credible intervals of 10% and 20% on either side of the mean

(b) Predictions with 10% and 20% error interval around the perfect prediction line

Figure 13: Prediction plots of model #5

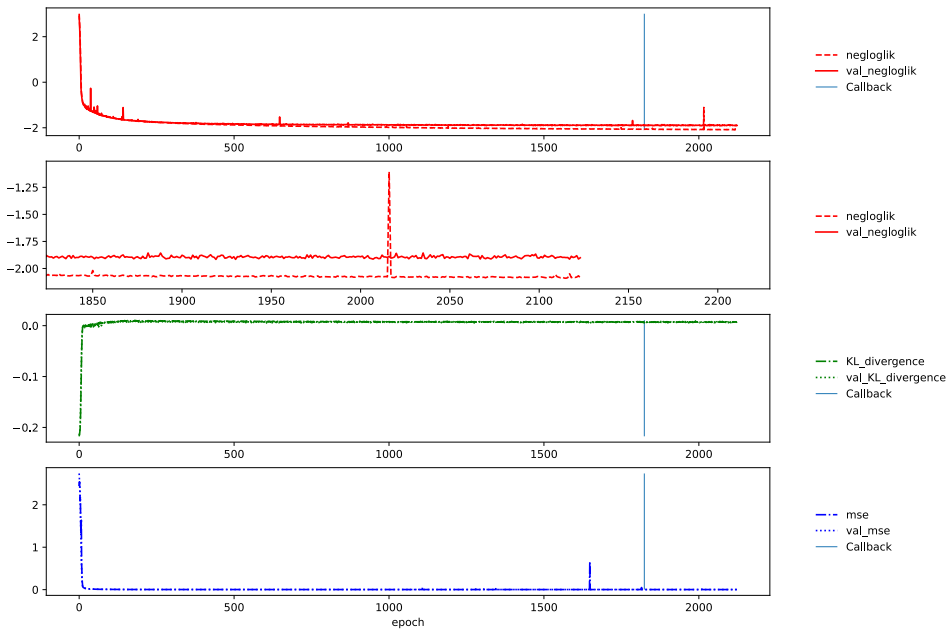


Figure 14: The history of the loss, KL-divergence and the mse for model #5

5. Conclusions

The following chapter contains the conclusions from this thesis project.

5.1 Discussion

From the toy example, both the benefit of having a distribution as the output and having distributions over the weights were apparent. In such a simple regression problem, decent results were given by a simple model. When looking at the predictions on insurance premiums, no clear conclusions could be made on which architecture gave the best result. The simplest model had the best loss but not the best KL-divergence nor mean square error. This could be due to local minimums of the different runs and could be addressed by doing multiple runs and averaging.

The training times varied quite a bit since the training did not run on a dedicated computer. So the results can vary due to other loads on the computer; this could be fixed by running the training on a dedicated server, which could make the training faster. During the training, a correlation between the size of the network and the training time per epoch was noticed. By this conclusion, a better measure would have been training time per epoch.

The stability of the training was another thing noticed; the models with fewer nodes per layer were more stable, meaning that they rarely diverged, whereas those with a higher number of nodes sometimes diverged. The reason for this could be the high number of parameters which is a result of more nodes per layer. Interestingly, the performance did not improve much by making the architecture more complex; one of the best performing models was the simplest one. This might relate to the low number of features. One other reason for the relatively good performance for a simple model could be that the insurance company probably has quite a simple model. As the predictions are now, they are not good enough to be used in production, i.e., their credible intervals are too wide. However, the models are not far off when considering the mean of the predictions.

An observation that has been made during this project is that BNNs have consistently created fatter tails. This consistency is key and one of the benefits of BNNs.

5.1.1 What could have been done better?

Many hours were spent trying to understand how the TensorFlow Probability API worked and how to use it. Thus, it was hard to find time to collect more features properly or work more on cleaning the data. These are the main points/ aspects that could have been done better, but also the parts that need the most amount of time.

The splitting of the data could also have been done differently. The prediction plots could have been made on a subset of the test set instead of making the test set an appropriate size for the plots.

5.2 Future Work

Following this thesis project, there are some areas where more work would potentially have some impact on ML for the stakeholder. Namely, more reliable data, hyperparameter tuning, and general adjustments.

5.2.1 Reliable Data

What this thesis boils down to in the end is the importance of reliable data, data that is saved in a more thoughtful and more structured way so that it can be used in the future for ML. What this essentially means is that when developing systems, an extra thought on how to save the data such that it can be used in the future is vital, if there is an interest in ML and AI, but also for Business intelligence (BI).

Damage and Claim Data

One feature that would have been beneficial is the damage done related to each insurance. This can also be generalized, meaning that better analysis can be made if the stakeholder would have more data on damages and claims. This relates to the fact that insurers use damage data to set the prices of insurances.

Outliers

When working with the data, it was cleaned from obvious outliers, but more work could have been done. For example, using Cook's distance to find more potential outliers would increase the data quality, leading to better-performing models.

5.2.2 Hyperparameter Tuning

Since there are so many hyperparameters that can be tweaked, it could be of interest to study how to automate the tuning process. There are many ways of doing this; studying, testing, and applying them could help in the process of having advanced ML-models.

Wide Distributions Outputs

As shown in the resulting prediction plot, see figure 13a, the distribution outputs are quite broad. This means that the 20% credible interval is almost as wide as the 40% interval. For that reason, the correct value is almost always included in the interval, but so are many other values. They are too wide to be helpful, and some hyperparameter tuning could potentially fix this.

5.2.3 Bayesian Neural Networks

BNNs are powerful, but more work is needed on inference methods to get better convergence of the distributions over the weights. The Flipout method is one good option. Investigating other methods may lead to better results. With some more work in this area, the future can be bright when it comes to BNNs.

5.3 Final Words

This work has shown that it is possible to use ML to predict insurance premiums. Furthermore, it has been demonstrated that BNNs are excellent in how the model can be seen as an infinite ensemble of models. It has also been shown that BNNs can be useful when it comes to producing credible intervals around the predictions. If the stakeholder would like to use ML on a larger scale than today, the focus should lie on securing and structuring data while saving all potential features. This would make it possible to leverage ML-models in many scenarios.

Bibliography

[1] Bishop, Christopher M. "Bayesian Neural Networks". In: *J. Braz. Comp. Soc* (1997). DOI: 10.1590/S0104-65001997000200006.

[2] Blei, David M., Kucukelbir, Alp, and McAuliffe, Jon D. "Variational Inference: A Review for Statisticians". In: *Journal of the American Statistical Association, Vol. 112* (2018). DOI: 10.1080/01621459.2017.1285773.

[3] Breiman, Leo. "Statistical Modeling: The Two Cultures". In: *Statistical Science* (2001).

[4] Cybenkot, G. "Approximation by Superpositions of a Sigmoidal Function". In: *Math. Control Signals Systems* (1989). DOI: 10.1007/BF02551274.

[5] DeepAI. *Feed Forward Neural Network*. May 2021. URL: https://deepai.org/machine-learning-glossary-and-terms/feed-forward-neural-network. Accessed: 2021-05-07.

[6] Etz, Alexander, Gronau, Quentin F., Edelsbrunner, Peter, and Dablander, Fabian. "How to become a Bayesian in eight easy steps: An annotated reading list". In: *Psychonomic Bulletin & Review* (2018).

[7] Henckaerts, Côté, Antonio, and Verbelen. *Boosting insights in insurance tariff plans with tree-based machine learning methods*. 2020. URL: https://arxiv.org/pdf/1904.10890.pdf.

[8] Hornik, Kurt. "Approximation capabilities of multilayer feedforward networks". In: *Neural Networks, Vol. 4* (1991). DOI: 10.1016/0893-6080(91)90009-T.

[9] Hüllermeier, Eyke and Waegeman, Willem. "Aleatoric and Epistemic Uncertainty in Machine Learning: An Introduction to Concepts and Methods". In: *Cornell University* (2020). URL: https://arxiv.org/pdf/1910.09457.pdf.

[10] Jostpin, Laurent Valentin, Buntine, Wray, Boussaid, Farid, Laga, Hamid, and Bennamoun, Mohammed. "Hands-on Bayesian Neural Networks - a Tutorial for Deep Learning Users". In: *ACM Computing Surveys* (2020).

[11] Keng, Brian. *Variational Bayes and The Mean-Field Approximation*. 2017. URL: https://bjlkeng.github.io/posts/variational-bayes-and-the-mean-field-approximation/. Accessed: 2021-03-09.

[12] Kingma, Diederik P., Salimans, Tim, and Welling, Max. "Variational Dropout Sparsifies Deep Neural Networks". In: *Cornell University* (2018). URL: https://arxiv.org/pdf/1506.02557.pdf.

[13] Kingma, Diederik P. and Welling, Max. "Auto-Encoding Variational Bayes". In: *Cornell University* (2018). URL: https://arxiv.org/pdf/1312.6114.pdf.

[14] Kuo, Kevin. "Individual Claims Forecasting with Bayesian Mixture Density Networks". In: *Cornell University* (2020). URL: https://arxiv.org/pdf/2003.02453.pdf.

[15] Lupton, Kuo anc. *Towards Explainability of Machine Learning Models in Insurance Pricing*. 2020. URL: https://arxiv.org/pdf/2003.10674.pdf.

[16] Mitros, John and Namee, Brian Mac. *On the Validity of Bayesian Neural Networks for Uncertainty Estimation*. 2019. URL: https://arxiv.org/pdf/1912.01530.pdf.

[17] Neiswanger, Willie. "Lecture 13 : Variational Inference: Mean Field Approximation". In: *Carnegie Mellon School of Computer Science* (2017). URL: https://www.cs.cmu.edu/~epxing/Class/10708-17/notes-17/10708-scribe-lecture13.pdf.

[18] Polson, Nicholas G. and Sokolov, Vadim O. "Deep Learning: A Bayesian Perspective". In: *Bayesian Analysis* (2017). DOI: 10.1214/17-BA1082.

[19] scikit-learn. *sklearn.preprocessing.StandardScaler*. May 2021. URL: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. Accessed: 2021-05-06.

[20] TensorFlow. *tfp.distributions*. Mar. 2021. URL: https://www.tensorflow.org/probability/api_docs/python/tfp/distributions. Accessed: 2021-03-12.

[21] TensorFlow. *tfp.layers.DenseFlipout*. Mar. 2021. URL: https://www.tensorflow.org/probability/api_docs/python/tfp/layers/DenseFlipout. Accessed: 2021-03-11.

[22] TensorFlow. *tfp.layers.DenseLocalReparameterization*. Mar. 2021. URL: https://www.tensorflow.org/probability/api_docs/python/tfp/layers/DenseLocalReparameterization. Accessed: 2021-03-11.

[23] TensorFlow. *tfp.layers.DenseReparameterization*. Mar. 2021. URL: https://www.tensorflow.org/probability/api_docs/python/tfp/layers/DenseReparameterization. Accessed: 2021-03-11.

[24] Transportstyrelsen. *Koder för fordonsuppgifter*. May 2021. URL: https://transportstyrelsen.se/sv/vagtrafik/Fordon/Fordonsregler/Koder-for-fordonsuppgifter/. Accessed: 2021-05-21.

[25] Walke, A. M. "On the Asymptotic Behaviour of Posterior Distributions". In: *Journal of the Royal Statistical Society* (1969).

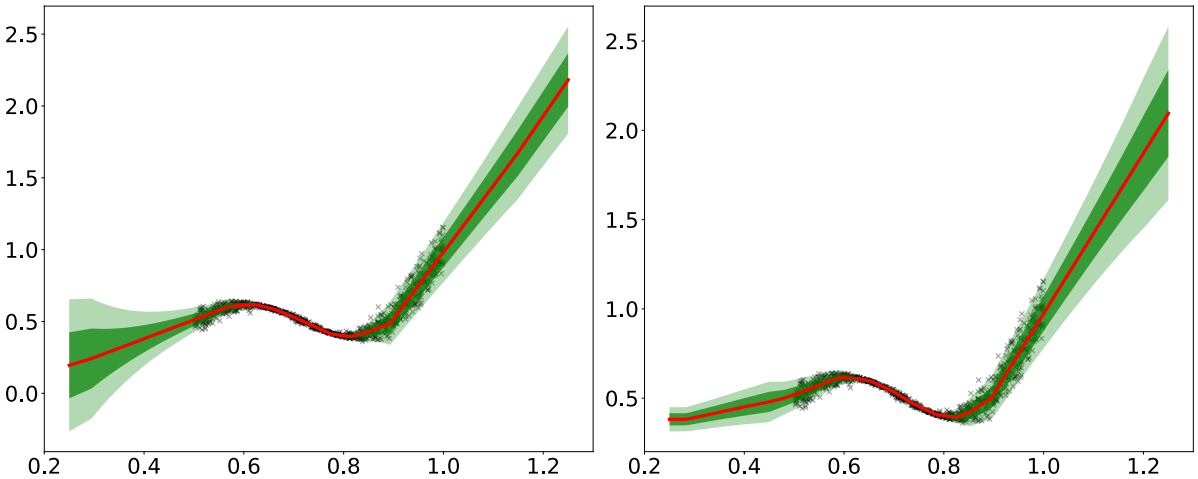
[26] Wen, Yeming, Vicol, Paul, Ba, Jimmy, Tran, Dustin, and Grosse, Roger. "Flipout: Efficient Pseudo-Independent Weight Perturbations on Mini-Batches". In: *Cornell University* (2018). URL: https://arxiv.org/pdf/1803.04386.pdf.

Appendix - Contents

A First Appendix **33**

B Second Appendix **34**

A. First Appendix



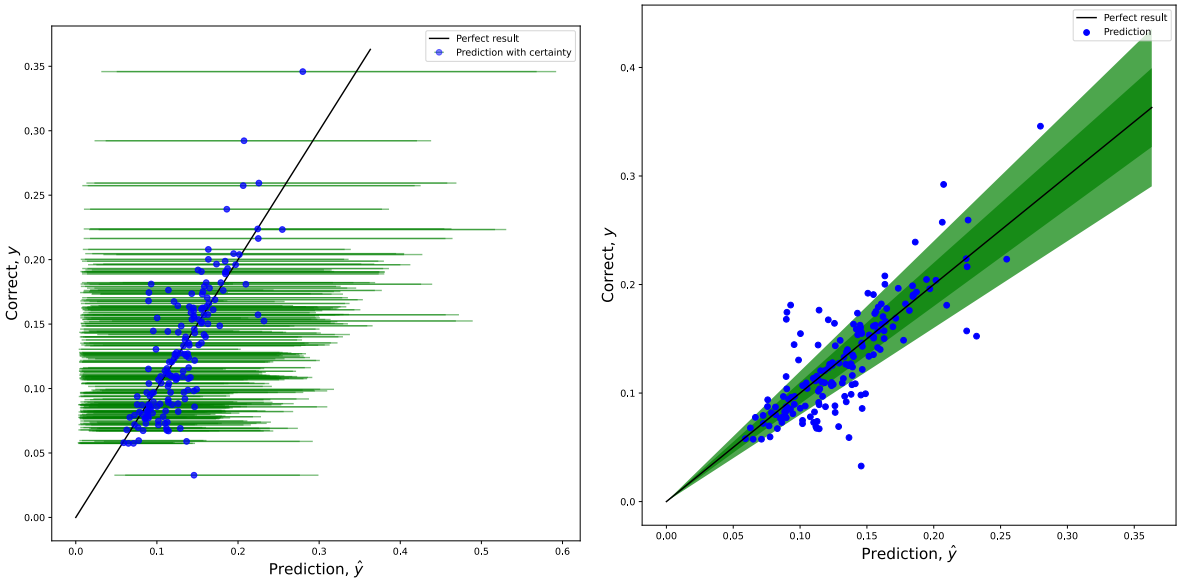
(a) Model # 6 with fatter left tail

(b) Model # 6 with fatter right tail

Figure 15: Two plots showing that the deterministic dense model can output larger credible intervals for the tails

These plots show examples of deterministic models with fatter tails, showing that the deterministic models can also have fat tails. Nevertheless, these models were not as consistent as the BNNs.

B. Second Appendix



(a) Prediction with credible intervals

(b) Predictions with 10% and 20% error interval around the perfect prediction line

Figure 16: Prediction plots of model #11

The plots in figure 16 show the output of a model with a higher number of parameters, which shows that the predictions are almost identical to the simple model displayed in the result, see figure 13.

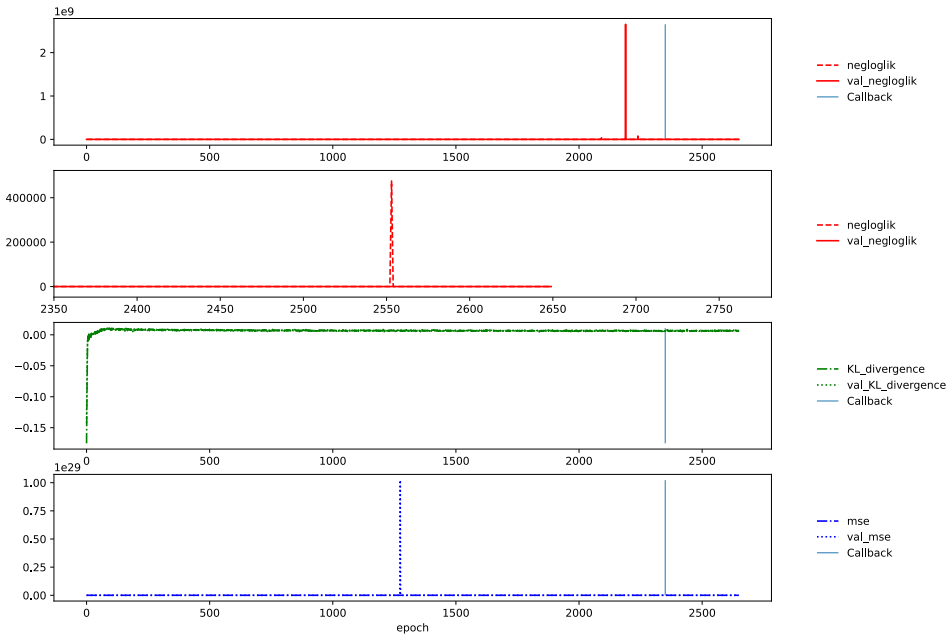


Figure 17: The history of the loss, KL-divergence and the mse for model #11

TRITA TRITA-SCI-GRU 2021:183

www.kth.se